

PHẦN 8.2

Dictionary<TKey, TValue> trong C#

Lưu trữ - Tra cứu - Xóa dữ liệu theo Key

Mục tiêu: hiểu rõ Key - Value, Add, ContainsKey, TryGetValue, Remove và cách đóng gói Dictionary trong class.

Tài liệu học và ôn tập trước khi làm bài StudentDirectory

Nội dung chính

- 1. Dictionary<TKey, TValue> là gì?
- 2. Key và Value; quy tắc Key duy nhất
- 3. Thêm dữ liệu bằng Add()
- 4. Truy cập value bằng key và rủi ro KeyNotFoundException
- 5. ContainsKey() và TryGetValue()
- 6. Xóa dữ liệu bằng Remove(key)
- 7. Cập nhật bằng indexer
- 8. Duyệt Dictionary với KeyValuePair
- 9. Count, Keys, Values và Clear()
- 10. So sánh Dictionary với List
- 11. Encapsulation với Dictionary
- 12. Ví dụ ProductDictionary hoàn chỉnh
- 13. Lỗi thường gặp và ghi nhớ nhanh
- 14. Bài tập StudentDirectory (không kèm đáp án)

Tài liệu không dùng LINQ để bạn hiểu rõ cách Dictionary hoạt động trước.

1. Dictionary<TKey, TValue> là gì?

Dictionary<TKey, TValue> là collection lưu dữ liệu theo từng cặp Key - Value. Mỗi Key dùng để xác định và lấy ra một Value tương ứng.

```
Dictionary<string, Product> products =  
    new Dictionary<string, Product>();
```

- string là kiểu của Key.
- Product là kiểu của Value.
- TKey và TValue là các tham số kiểu generic.

Có thể hình dung: "P01" -> Product Laptop, "P02" -> Product Mouse.

Cú pháp tổng quát:

```
Dictionary<TKey, TValue> dictionary =  
    new Dictionary<TKey, TValue>();
```

Ví dụ phổ biến:

```
Dictionary<string, Student> students;  
Dictionary<int, string> names;  
Dictionary<string, Product> products;
```

2. Key và Value

Key là khóa dùng để tra cứu. Value là dữ liệu được lưu tại khóa đó. Trong một Dictionary, Key phải duy nhất nhưng Value có thể trùng nhau.

```
students.Add("SV01", "Nguyen Van An");
students.Add("SV02", "Tran Thi Binh");
```

Thêm Key trùng:

```
students.Add("SV01", "Le Van Cuong"); // lỗi
```

Kết quả thường là ArgumentException vì Dictionary không cho phép hai phần tử có cùng Key.

Value có thể trùng:

```
students.Add("SV03", "Nguyen Van An"); // hợp lệ
```

Quy tắc: Key duy nhất; Value không bắt buộc duy nhất.

3. Thêm dữ liệu bằng Add()

```
Product product =
    new Product("P01", "Laptop", 20_000_000m);

products.Add(product.Id, product);
```

Ở đây product.Id là Key, còn product là Value.

Nên tránh Key và Id không đồng nhất:

```
products.Add(
    "P99",
    new Product("P02", "Mouse", 500_000m));
```

Cách an toàn: tạo Product trước, sau đó dùng products.Add(product.Id, product).

4. Truy cập Value bằng Key

```
Product product = products["P01"];
product.DisplayInfo();
```

Cú pháp dấu ngoặc vuông được gọi là indexer. Dictionary dùng Key thay vì index số như List.

Rủi ro khi Key không tồn tại:

```
Product product = products["P99"];
```

Nếu "P99" không tồn tại, chương trình phát sinh `KeyNotFoundException`.

Không nên dùng `dictionary[key]` khi chưa chắc Key tồn tại.

5. ContainsKey() và TryGetValue()

`ContainsKey()` dùng khi chỉ cần biết Key có tồn tại hay không.

```
if (products.ContainsKey("P01"))
{
    Product product = products["P01"];
    product.DisplayInfo();
}
```

TryGetValue() vừa kiểm tra, vừa lấy Value:

```
if (products.TryGetValue(
    "P01",
    out Product? product))
{
    product.DisplayInfo();
}
else
{
    Console.WriteLine("Product not found.");
}
```

Ý nghĩa của out:

- Method nhận Key cần tìm.
- Nếu tìm thấy, Value được đưa ra biến `product`.
- Method trả về `true` nếu tìm thấy, `false` nếu không tìm thấy.

So sánh:

Tình huống	Nên dùng	Lý do
Chỉ cần kiểm tra Key	<code>ContainsKey()</code>	Không cần lấy Value
Cần lấy Value	<code>TryGetValue()</code>	Kiểm tra và lấy trong một lần

6. Xóa dữ liệu bằng Remove(key)

```
bool removed = products.Remove("P01");
```

- true: Key tồn tại và phần tử đã được xóa.
- false: Key không tồn tại.

```
if (products.Remove("P01"))
{
    Console.WriteLine("Product removed successfully.");
}
else
{
    Console.WriteLine("Product not found.");
}
```

Khác với List, Dictionary có thể xóa trực tiếp bằng Key mà không cần tìm object trước.

7. Cập nhật bằng indexer

```
products["P01"] =
    new Product(
        "P01",
        "Gaming Laptop",
        30_000_000m);
```

- Nếu Key đã tồn tại: Value cũ bị thay thế.
- Nếu Key chưa tồn tại: một phần tử mới được thêm.

Khác nhau giữa Add và indexer:

Cách viết	Key đã tồn tại	Key chưa tồn tại
Add(key, value)	Ném exception	Thêm mới
dictionary[key] = value	Ghi đè	Thêm mới

Khi không muốn ghi đè nhầm, nên kiểm tra Key trước khi cập nhật.

8. Duyệt Dictionary với KeyValuePair

```
foreach (
    KeyValuePair<string, Product> item
    in products)
{
    Console.WriteLine($"Key: {item.Key}");
    item.Value.DisplayInfo();
}
```

- item.Key là Key của phần tử.
- item.Value là Value của phần tử.

Duyệt riêng Keys:

```
foreach (string id in products.Keys)
{
    Console.WriteLine(id);
}
```

Duyệt riêng Values:

```
foreach (Product product in products.Values)
{
    product.DisplayInfo();
}
```

9. Count và Clear()

```
Console.WriteLine(products.Count);
products.Clear();
```

- Count cho biết số lượng cặp Key - Value hiện có.
- Clear() xóa toàn bộ phần tử nhưng Dictionary vẫn tồn tại.
- Sau Clear(), vẫn có thể Add() dữ liệu mới.

10. So sánh List và Dictionary

Tiêu chí	List<T>	Dictionary<TKey, TValue>
Cách lưu	Danh sách phần tử	Cặp Key - Value
Truy cập	Index 0, 1, 2...	Theo Key
Trùng dữ liệu	Có thể	Key không được trùng
Tìm theo Id	Thường phải duyệt	Tìm trực tiếp theo Key
Phù hợp	Hiển thị tuần tự	Tra cứu theo mã

List phù hợp khi cần một danh sách tuần tự; Dictionary phù hợp khi cần tra cứu theo một khóa duy nhất.

11. Encapsulation với Dictionary

Không nên để Dictionary là public vì code bên ngoài có thể Clear(), Add() hoặc thay đổi dữ liệu mà không đi qua validation của class.

```
public Dictionary<string, Product> Products
{
    get;
    set;
}
```

Cách tốt hơn:

```
private readonly Dictionary<string, Product>
    _products;
```

- private: bên ngoài không truy cập trực tiếp.
- readonly: field không bị gán sang một Dictionary khác sau constructor.
- Dữ liệu chỉ thay đổi qua AddProduct, FindProductById, RemoveProductById, DisplayProducts.

12. Ví dụ hoàn chỉnh: ProductDictionary

Class Product:

```
public class Product
{
    public string Id { get; }
    public string Name { get; }
    public decimal Price { get; }

    public Product(
        string id,
        string name,
        decimal price)
    {
        if (string.IsNullOrEmpty(id))
        {
            throw new ArgumentException(
                "Id cannot be null or whitespace.",
                nameof(id));
        }

        if (string.IsNullOrEmpty(name))
        {
            throw new ArgumentException(
                "Name cannot be null or whitespace.",
                nameof(name));
        }

        if (price <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(price),
                price,
                "Price must be greater than zero.");
        }

        Id = id.Trim();
        Name = name.Trim();
        Price = price;
    }

    public void DisplayInfo()
    {
        Console.WriteLine(
            $"Id: {Id}, Name: {Name}, " +
            $"Price: {Price:N0} VND");
    }
}
```

12.1. ProductDictionary - khai báo và constructor

```
public class ProductDictionary
{
    public string Name { get; }

    private readonly Dictionary<string, Product>
        _products;

    public ProductDictionary(string name)
    {
        if (string.IsNullOrEmpty(name))
        {
            throw new ArgumentException(
                "Name cannot be null or whitespace.",
                nameof(name));
        }

        Name = name.Trim();
        _products =
            new Dictionary<string, Product>();
    }
}
```

12.2. AddProduct()

```
public void AddProduct(Product product)
{
    if (product == null)
    {
        throw new ArgumentNullException(
            nameof(product),
            "Product cannot be null.");
    }

    if (_products.ContainsKey(product.Id))
    {
        throw new InvalidOperationException(
            $"Product with Id {product.Id} " +
            "already exists.");
    }

    _products.Add(product.Id, product);
}
```

12.3. Find, Remove và Display

```

public Product? FindProductById(string id)
{
    if (string.IsNullOrEmpty(id))
    {
        throw new ArgumentException(
            "Id cannot be null or whitespace.",
            nameof(id));
    }

    string normalizedId = id.Trim();

    if (_products.TryGetValue(
        normalizedId,
        out Product? product))
    {
        return product;
    }

    return null;
}

public bool RemoveProductById(string id)
{
    if (string.IsNullOrEmpty(id))
    {
        throw new ArgumentException(
            "Id cannot be null or whitespace.",
            nameof(id));
    }

    return _products.Remove(id.Trim());
}

public void DisplayProducts()
{
    Console.WriteLine($"Catalog: {Name}");

    if (_products.Count == 0)
    {
        Console.WriteLine(
            "The product dictionary is empty.");
        return;
    }

    foreach (
        KeyValuePair<string, Product> item
        in _products)
    {
        Console.WriteLine(
            $"Dictionary key: {item.Key}");
        item.Value.DisplayInfo();
    }
}

```

13. Lỗi thường gặp

Lỗi 1 - Thêm Key trùng:

```
_products.Add(product.Id, product);
_products.Add(product.Id, product); // lỗi
```

- Giải pháp: kiểm tra ContainsKey() trước khi Add().

Lỗi 2 - Truy cập Key không tồn tại:

```
Product product = _products["P99"];
```

- Giải pháp: dùng TryGetValue().

Lỗi 3 - Key và Product.Id khác nhau:

```
_products.Add("P01", new Product("P02", "Mouse", 500_000m));
```

- Giải pháp: dùng _products.Add(product.Id, product).

Lỗi 4 - Public Dictionary:

- Code bên ngoài có thể Clear() hoặc Add() mà không qua validation.
- Giải pháp: private readonly Dictionary và các method quản lý.

Ghi nhớ nhanh:

Cần lấy Value -> TryGetValue(). Chỉ kiểm tra Key -> ContainsKey(). Xóa -> Remove(key).

14. Câu hỏi tự kiểm tra

1. Dictionary lưu dữ liệu theo cấu trúc nào?
2. TKey và TValue đại diện cho điều gì?
3. Key có được trùng không? Value có được trùng không?
4. Vì sao không nên lấy dữ liệu trực tiếp bằng dictionary[key] khi chưa chắc Key tồn tại?
5. ContainsKey() khác TryGetValue() ở điểm nào?
6. Remove(key) trả về giá trị gì?
7. Indexer dictionary[key] = value khác Add(key, value) như thế nào?
8. KeyValuePair<TKey, TValue> dùng trong trường hợp nào?
9. Vì sao Dictionary nên là private readonly?
10. Khi nào nên chọn Dictionary thay vì List?

15. Bài tập 8.2 - StudentDirectory

Bài tập không kèm đáp án. Hãy tự viết code và gửi lại để được đánh giá.

Class Student

- Thuộc tính: Id, FullName, Gpa.
- Id không rỗng.
- FullName không rỗng.
- Gpa từ 0 đến 10.
- Property không dùng public set.
- Có method DisplayInfo().

Class StudentDirectory

- Property Name.

```
private readonly Dictionary<string, Student>  
    _students;
```

- Constructor validation Name và khởi tạo Dictionary rỗng.

Các method bắt buộc

```
public void AddStudent(Student student)  
public Student? FindStudentById(string id)  
public bool RemoveStudentById(string id)  
public void DisplayStudents()
```

- AddStudent: kiểm tra null, không cho trùng Id, dùng student.Id làm Key.
- FindStudentById: validation id và dùng TryGetValue().
- RemoveStudentById: validation id, trả về kết quả Remove().
- DisplayStudents: kiểm tra rỗng và duyệt KeyValuePair<string, Student>.

16. Yêu cầu trong Program.cs

11. Tạo StudentDirectory.
12. Thêm ít nhất ba sinh viên.
13. Hiển thị toàn bộ sinh viên.
14. Tìm sinh viên có Id = "SV02".
15. Hiển thị kết quả tìm kiếm.
16. Thử tìm một Id không tồn tại.
17. Xóa sinh viên có Id = "SV01".
18. Hiển thị kết quả xóa.
19. Hiển thị lại Dictionary sau khi xóa.
20. Thử thêm một sinh viên có Id bị trùng.
21. Dùng try-catch phù hợp.

Bắt buộc sử dụng

- Dictionary<string, Student>
- Add()
- ContainsKey()
- TryGetValue()
- Remove()
- KeyValuePair<string, Student>

Chưa sử dụng LINQ trong bài tập này.

Checklist trước khi nộp bài

- Không dùng public set cho Id, FullName, Gpa.
- Có validation chuỗi và Gpa.
- Dictionary là private readonly.
- Không thêm Key trùng.
- Find dùng TryGetValue().
- Remove dùng Remove(key).
- Display duyệt KeyValuePair.
- Có kiểm tra Dictionary rỗng.
- Catch đúng loại exception.
- Code trong Program kiểm tra cả trường hợp thành công và thất bại.